

CMPE-679 Lab 1

TA: Ishan Renjen

January 21, 2026

Lab 1 Paper

1 Introduction

This lab is intended to be an introduction to Pytorch and NLP with a Binary Sentiment Classification project. First, the IMDb dataset is downloaded and a pretrained gLoVe embedding is downloaded for processing the text and creating embeddings. An LSTM model will be created to input the embedding generated from the text data, and predict a single value. The model will be trained on a binary value label representing a positive or negative review. The model will be evaluated in terms of test/validation loss, accuracy on the test set, and metrics of your choice.

2 Data Preprocessing

The dataset used for this project is the IMDb dataset, containing a set of 25k strings per train/test split labeled with a 0 for negative sentiment and a 1 for positive sentiment. These strings, with each being a review, are processed and turned into embeddings to be used to train the model. The general steps from the paper to do so are as follows:

- remove punctuation
- eliminate stopwords
- lowercase all text
- create a dictionary of unique words, assigning an index to a word to be used in the conversion of review $\rightarrow$ vector

These steps, and the paper in general serve as a starting point - the goal is to have high accuracy in classifying reviews from the IMDb database - **the steps do not need to be followed exactly provided you have a valid justification for doing so.** Ensure that you only implement the preprocessing steps that will lead to a higher accuracy. In addition, there are many ways to do the same steps, which can be equally valid. One suggestion is to use NLTK, which contains a lot of what you will need to do all of the above.

Once the vocabulary is built and the text has been preprocessed, the next step is tokenization. Tokenization is the process of breaking up a larger string into multiple substrings to be converted into embeddings. There are multiple methods of doing so, which will be discussed in a later lab. For this lab, use a tokenizer from NLTK or Hugging Face. **Make sure to discuss why you chose the tokenizer you did.** The tokenization method will correspond to a higher accuracy due to the representation of the input in the embedding. Keep in mind that you cannot process a variable amount of tokens - **discuss how you limit the number of tokens or process the strings so that they are all the same length.**

Now that tokenization is done, the list of tokens will become an embedding. The embedding is pretrained from <http://nlp.stanford.edu/data/glove.6B.zip> as seen in that link. from that zip, use `"/content/glove.6B.300d.txt"`. This matrix correlates each word in its vocabulary to a 300-dimensional vector. This is what will be used to convert the tokens to vectors which can be processed by the model.

One thing to note here is that you want to be sure that your vocabulary indices will correlate to your embedding matrix rows.

The implementation for this part will vary - depending on how you do it, you may not need a dataset class but there will be some code such as a collate function that is not explained in class or homeworks. Make sure to read the Pytorch documentation for this, it is helpful. A good starting point is read through the DataLoader documentation which points you towards everything you will need for that.

3 Model

The next part of the lab is writing the model. This consists of the (frozen) embedding matrix you previously imported and ordered to match your vocabulary, a BiLSTM layer, and the pooling layers described in the paper. This diagram is shown in Figure 1 below.

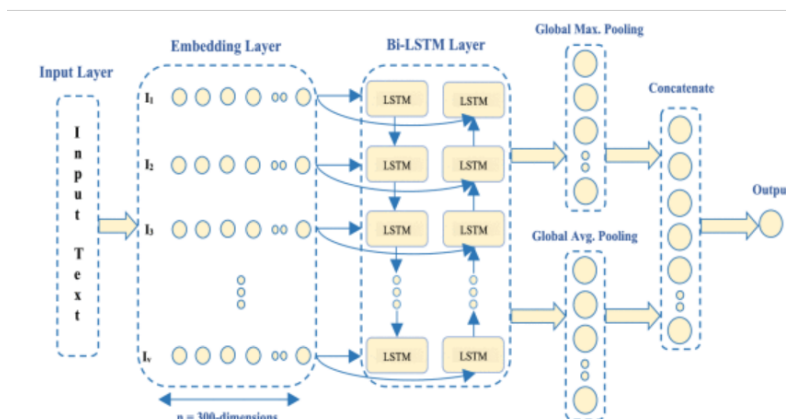


Figure 1: Model Diagram

In Figure 1, you can see the input tokens going into an embedding layer, which is directly fed into the BiLSTM layer. Then, these are fed into a set of pooling layers which are then concatenated and then projected from the concatenated output to a single value.

The first general model for processing text/sequential data is an RNN. An RNN processes a sequential input with a shared set of parameters in the hidden layer between the input and the output. The hidden layer stores the previous input for future predictions, acting as "memory". RNN's come with problems such as vanishing/exploding gradients with increasingly large models due to repeated activation functions and smaller weights. To solve this, the LSTM is introduced.

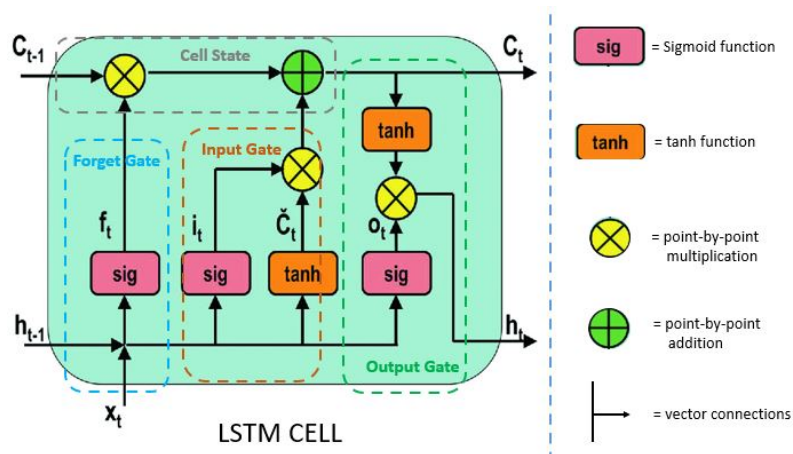


Figure 2: LSTM Diagram

The LSTM introduces multiple gates, controlling what is stored in the internal memory and what the output will be. This solves the vanishing gradient problem because of the gradients flowing through the gates being better regulated than direct multiplication of the weights in an RNN. A BiLSTM expands on this by feeding data through both forward and backwards, enabling it to find dependencies in the entire sequence.

The pooling layers take the output of the LSTM, and perform 2 different pooling operations before concatenating and projecting it into a single value. The global max pooling will determine the maximum value, and the global average pooling will determine the average value.

4 Training/Evaluation

For training this model, you are responsible for writing a functional training/evaluation loop. Read over PyTorch documentation and use your homework as a reference. For evaluation, the basic metric in question will be accuracy. **Make sure when evaluating accuracy, turn off gradients on the computation graph and put the model in eval() mode.** Otherwise, the evaluation data will be used for training which defeats the purpose. In addition, you will pick metrics of your choice to evaluate this model. **Make sure to pick metrics that make sense in this context and give a justification as to why each one was picked, as well as its downsides.** Save the best model for your project submission. A useful tool for colab or local computers is gradient checkpointing, or saving the current state of the model at specified checkpoints so if training stops for some reason you can continue from there. This can potentially save you time and frustration.

5 Deliverables

Google Drive link to a folder with:

- Jupyter Notebook (.ipynb) with all the code for the assignment - **The code should be fully functional or no points will be given.**
- Model file (.pth) saved from the best-performing epoch in training
- Short informal report with lab description, results, issues encountered, potential improvements - **The report must be included or no points will be given.**

6 Grading

- **Functional Jupyter Notebook and Model:** 50 pts
- **Accuracy of 86% or more on testing and validation set:** 20 points
 - Accuracy of 80% or more: 10 points
 - Below 80%: 0 points
- **Report:** 20 points
 - Summary of lab (explain purpose of lab, data, model): 5 points
 - Testing and Validation loss over all epochs in some manner: 10 points
 - Conclusion (explain issues, potential improvements, etc): 5 points
- **Questions in Report:** 10 points
 - Q1:** Feedback on lab: anything you would like to see changed: 5 points
 - Q2:** Research evaluation metrics for the model and implement 2+ evaluation metrics, state results: 5 points